

Hadoop & HDFS

Interview Prep Guide — Concepts, Architecture & Commands



Big Data Engineering Reference — Hadoop Ecosystem, HDFS Internals, and the CLI Commands You'll Actually Use

What is Hadoop?

Apache Hadoop is an open-source framework for storing and processing very large datasets across clusters of commodity hardware. It's built on a simple idea: instead of one huge, expensive machine, use many cheap machines working together — and instead of moving data to the computation, **move the computation to the data**.

The Core Problem Hadoop Solves

Traditional systems struggle once data outgrows a single machine's storage or processing power. Hadoop solves this with **distributed storage** (HDFS) and **distributed processing** (MapReduce / YARN), scaling horizontally by adding more machines rather than buying bigger ones.

The Four Core Modules

HDFS

Distributed file system for storing massive datasets across many machines, with built-in fault tolerance via replication.

YARN

"Yet Another Resource Negotiator" — manages cluster resources (CPU, memory) and schedules jobs across nodes.

MapReduce

Programming model that processes data in two phases — Map (transform) and Reduce (aggregate) — in parallel across the cluster.

Hadoop Common

Shared libraries and utilities that the other modules depend on.

The Wider Ecosystem

Tool	Purpose
Hive	SQL-like querying over data stored in HDFS (translates SQL to MapReduce/Tez/Spark jobs)
Pig	Scripting language (Pig Latin) for data transformation pipelines
HBase	NoSQL, column-oriented database built on top of HDFS for real-time read/write access
Sqoop	Transfers data between HDFS and relational databases (import/export)
Flume	Ingests streaming/log data into HDFS
Zookeeper	Coordination service — manages configuration, naming, and leader election (e.g. NameNode HA)
Spark	Fast, in-memory alternative/complement to MapReduce; often reads data directly from HDFS
Oozie	Workflow scheduler for chaining Hadoop jobs

💡 **Interview framing:** Think of Hadoop as an *ecosystem*, not a single tool. HDFS is the storage layer, YARN is the resource manager, and MapReduce (or Spark) is the processing engine. Most real answers should mention how these three fit together.

HDFS — Hadoop Distributed File System

HDFS is the storage layer of Hadoop. It splits large files into fixed-size **blocks** (default 128 MB in Hadoop 2/3, was 64 MB in Hadoop 1), spreads those blocks across many machines, and **replicates** each block (default 3x) so the cluster survives hardware failure without losing data.

Core Design Principles

Principle	What it means
Write-once, read-many	Files are typically written once and read many times — optimized for batch analytics, not frequent updates
Large block size	Fewer, bigger blocks (128 MB+) reduce the overhead of tracking metadata for huge datasets
Fault tolerance via replication	Each block is copied to multiple DataNodes (default replication factor = 3)
Rack awareness	Replicas are placed across different racks so a single rack failure doesn't lose data
Move computation, not data	Processing frameworks (MapReduce/Spark) run tasks on the node where the data block already lives

Architecture — Master / Slave Model

- NameNode** — the **master**. Stores metadata only: file names, block locations, permissions, directory tree. Keeps this in RAM for speed. Does **not** store actual file data.
- DataNode** — the **slave/worker**. Stores the actual data blocks on local disk. Sends periodic "heartbeats" and block reports to the NameNode.
- Secondary NameNode** — **not** a hot backup. Periodically merges the NameNode's edit logs with its filesystem image (checkpointing) to keep metadata recovery fast. In modern Hadoop, HA setups use a **Standby NameNode** instead.
- Client** — talks to the NameNode to get block locations, then reads/writes data blocks directly to/from the relevant DataNodes.

How a Write Works

- Client asks the NameNode where to write the file → NameNode returns a list of DataNodes for each block.
- Client streams the block to the first DataNode, which forwards it to the second, which forwards it to the third — a **replication pipeline**.
- Each DataNode acknowledges back up the pipeline once the write is confirmed.
- NameNode updates its metadata once all replicas are written.

How a Read Works

- Client asks the NameNode for the block locations of a file.
- NameNode returns the DataNodes holding each block, ordered by proximity to the client.
- Client reads blocks directly from the nearest DataNode — the NameNode is not involved in the actual data transfer.

⚠ **Classic interview question:** "What happens if the NameNode fails?" — Without HA, the whole cluster becomes unavailable because there's a single point of metadata. Modern Hadoop solves this with **NameNode High Availability** (Active + Standby NameNode, coordinated via Zookeeper and a shared edit log via JournalNodes).

⚠ **Classic interview question:** "What is the small files problem?" — Every file, directory, and block consumes ~150 bytes of NameNode RAM. Millions of tiny files exhaust NameNode memory and hurt performance — HDFS is designed for a smaller number of large files, not many small ones. Common fixes: Hadoop Archives (HAR), SequenceFiles, or combining files before ingesting.

HDFS Commands for Big Data Engineering

All HDFS shell commands follow the pattern `hdfs dfs -<command> [options] [path]` (the older `hadoop fs -<command>` syntax still works too). Below is a practical set grouped by what you'd actually do on the job.

File & Directory Basics

Command	What it does
<code>hdfs dfs -ls /path</code>	List files/directories
<code>hdfs dfs -ls -R /path</code>	List recursively
<code>hdfs dfs -mkdir /path/newdir</code>	Create a directory
<code>hdfs dfs -mkdir -p /a/b/c</code>	Create nested directories
<code>hdfs dfs -rm /path/file.txt</code>	Delete a file
<code>hdfs dfs -rm -r /path/dir</code>	Delete a directory recursively
<code>hdfs dfs -rm -skipTrash /path/file</code>	Delete permanently, bypassing Trash
<code>hdfs dfs -mv /src /dst</code>	Move / rename a file or directory
<code>hdfs dfs -cp /src /dst</code>	Copy within HDFS
<code>hdfs dfs -touchz /path/file</code>	Create an empty file

Moving Data In & Out (Local ⇄ HDFS)

Command	What it does
<code>hdfs dfs -put localfile.csv /hdfs/path/</code>	Upload a local file to HDFS
<code>hdfs dfs -copyFromLocal localfile /hdfs/path/</code>	Same as -put, explicit about direction
<code>hdfs dfs -get /hdfs/path/file.csv .</code>	Download a file from HDFS to local disk
<code>hdfs dfs -copyToLocal /hdfs/path/file.csv .</code>	Same as -get, explicit about direction
<code>hdfs dfs -moveFromLocal local.csv /hdfs/path/</code>	Upload then delete the local copy
<code>hdfs dfs -getmerge /hdfs/dir/ local_merged.csv</code>	Merge many HDFS files into one local file (great for pulling MapReduce/Spark part-files)

Inspecting Files & Data

Command	What it does
<code>hdfs dfs -cat /path/file.txt</code>	Print file contents to screen
<code>hdfs dfs -tail /path/file.txt</code>	Show the last 1KB of a file (good for logs)
<code>hdfs dfs -du -h /path</code>	Show disk usage of files (human-readable)
<code>hdfs dfs -du -s -h /path</code>	Show total (summarized) size of a directory
<code>hdfs dfs -df -h</code>	Show free/used space of the whole HDFS filesystem
<code>hdfs dfs -count /path</code>	Count files, directories, and total size under a path
<code>hdfs dfs -stat "%b %o %r" /path/file</code>	Show block size, replication factor, owner, etc.

Permissions & Ownership

Command	What it does
<code>hdfs dfs -chmod 755 /path/file</code>	Change file permissions
<code>hdfs dfs -chown user:group /path/file</code>	Change file owner and group
<code>hdfs dfs -chgrp group /path/file</code>	Change group ownership only

Cluster Administration & Health

Command	What it does
<code>hdfs dfsadmin -report</code>	Cluster-wide summary — capacity, DataNodes, live/dead nodes
<code>hdfs dfsadmin -safemode enter</code>	Put NameNode in safe mode (read-only, no writes)
<code>hdfs dfsadmin -safemode leave</code>	Take NameNode out of safe mode
<code>hdfs dfs -setrep -w 2 /path/file</code>	Change replication factor of a file (here, to 2)
<code>hdfs fsck /path -files -blocks</code>	Check filesystem health — file integrity, block locations, corruption
<code>hdfs balancer</code>	Rebalance data evenly across DataNodes
<code>hdfs dfsadmin -refreshNodes</code>	Refresh the list of allowed/excluded DataNodes
<code>hdfs version</code>	Show installed Hadoop version

💡 **Interview tip:** If asked "how would you check if HDFS is healthy," the strongest answer combines `hdfs dfsadmin -report` (cluster capacity/nodes) with `hdfs fsck /` (block-level integrity check) — don't just name one command.

💡 **Interview tip:** Know the difference between `-du` (per-file/directory sizes) and `-df` (overall filesystem capacity) — a common quick-fire question.

Rapid-Fire Interview Q&A

Q: What is the default block size in HDFS?

128 MB in Hadoop 2.x/3.x (was 64 MB in Hadoop 1.x) — configurable via `dfs.blocksize`.

Q: What is the default replication factor?

3 — configurable per-file or cluster-wide via `dfs.replication`.

Q: Why is the block size so large compared to a typical OS file system?

To minimize the ratio of seek time to transfer time and to keep NameNode metadata (which tracks every block) manageable for petabyte-scale data.

Q: What's the difference between HDFS and a traditional distributed file system (e.g. NFS)?

HDFS is optimized for large sequential reads/writes of huge files with built-in replication and fault tolerance, and it favors throughput over low latency — it's not meant for many small random reads/writes.

Q: What does the NameNode actually store?

Only metadata — the filesystem namespace, directory tree, and block-to-DataNode mapping. Block-to-DataNode mapping is rebuilt in memory from DataNode block reports at startup, not stored on disk.

Q: What is rack awareness?

HDFS's strategy of placing replicas across different racks (not just different nodes) so that a rack-level failure (e.g. a switch going down) doesn't cause data loss.

Q: How does HDFS achieve fault tolerance?

Through block replication (default 3 copies), rack-aware placement, DataNode heartbeats/health checks, and automatic re-replication when a DataNode fails or a block becomes under-replicated.

Q: What happens when a DataNode goes down?

The NameNode detects the missing heartbeat, marks the DataNode as dead, and schedules re-replication of the blocks it held onto other healthy DataNodes to restore the target replication factor.

Q: What's the difference between MapReduce and YARN?

MapReduce is a processing model/programming paradigm; YARN is the resource manager that schedules and allocates cluster resources for MapReduce (and other frameworks like Spark) to run on.

Q: When would you NOT use HDFS?

For low-latency random access to small records (use HBase instead), for frequently updated/mutable data, or for a small number of small files where the overhead isn't justified.

Quick Recap

- **Hadoop** = distributed storage (HDFS) + resource management (YARN) + distributed processing (MapReduce/Spark), plus a large ecosystem (Hive, HBase, Sqoop, etc.)
- **HDFS** = fault-tolerant distributed storage using large blocks, replication, and a NameNode/DataNode master-slave architecture
- **Commands** = `hdfs dfs -ls/-put/-get/-cat/-du/-rm` for daily file work, `hdfs dfsadmin -report` and `hdfs fsck` for health checks
- Be ready to explain NameNode failure handling, the small-files problem, and how reads/writes flow through the cluster — these come up constantly